

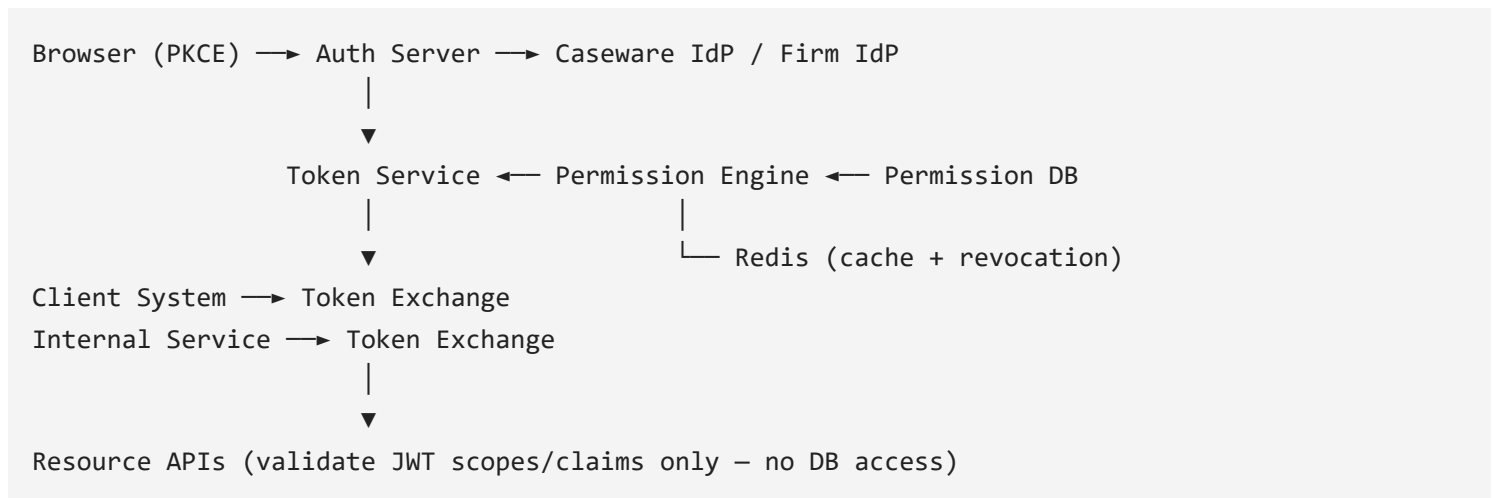
Collaborate Authorization Layer — Design Document

Author: Carlos Sanchez

Context: Caseware Collaborate — OAuth2/OIDC authorization layer for multi-tenant engagement workspaces.

1. High-Level Architecture

Collaborate exposes an **Authorization Layer** between clients and downstream resource APIs (Document Service, Comments Service, Financial Data API). The layer does not replace Caseware's central identity provider (IdP) or firm-federated IdPs; it **issues and validates access tokens** enriched with workspace permissions and firm policy.



Use Case 1: Login (Authorization Code + PKCE)

- **Firm staff** authenticate against Caseware's central OIDC IdP.
- **External client users** may authenticate via a **firm-federated SAML/OIDC IdP**, scoped to that firm's workspaces.
- Per-firm **OAuth client configuration** (`client_id`, redirect URIs, allowed IdP) routes login to the correct upstream provider.
- After upstream authentication, Collaborate's Token Service **enriches** the session with workspace roles and permission version before issuing access tokens.

Use Case 2: Permission Checking (Fast, Revocable)

Permissions combine three layers stored in Collaborate's database:

Layer	Example
Workspace role	owner / contributor / viewer
Resource override	single document shared with one external user
Firm policy	firm-wide restrictions on external access

Fast path: Resource APIs validate JWT claims only (scope, workspace_id, perm_version). A Redis cache keyed by (userId, workspaceId, resourceId, permVersion) avoids DB round-trips for repeated checks at token issuance and optional gateway-level enrichment.

Revocation within seconds: Each permission change increments a workspace **permission version**. Tokens carry perm_version; Redis publishes invalidation events on DB changes. APIs compare token version against cached current version — mismatch yields 403 without waiting for token expiry. Short access-token TTL (5–15 min) limits stale-allow window for long-lived sessions (e.g., collaborative editing).

Use Case 3: On-Behalf-Of Authorization

Two delegation paths, both via **RFC 8693 Token Exchange**:

1. **Client system** → **Collaborate API** on behalf of an employee (pull engagement data into internal systems).
2. **Internal Collaborate service** → **Caseware API** on behalf of the user who triggered an action (audit attribution preserved).

Confused deputy prevention:

- Validate **actor token** (calling client/service) is a registered integration with token-exchange grant.
- Issue a **narrower** token: restricted aud, reduced scope, explicit sub (delegated user), act/actor claim for audit.
- Reject exchanges where requested scope exceeds actor's or subject's effective permissions.
- Bind tokens to client_id + resource identifiers via authorization_details where applicable.

Assumption: Credential storage, MFA, and full IdP implementation are out of scope — upstream IdPs are external dependencies.

2. Implementation Plan

Phase	Scope	Deliverables
1 — Token validation contract	JWT bearer middleware, scope/claim policies per API	Protected resource endpoints (this exercise, Option A)

2 — Login & federation	OIDC middleware, per-firm client registry, PKCE, federated IdP discovery	Interactive login flows
3 — Permission engine	DB schema, Redis cache-aside, change-event hooks, token enrichment at issuance	Sub-second permission checks; version-based revocation
4 — On-behalf-of	Token exchange endpoint, actor validation, structured audit logs	Delegated tokens with narrowed scope

AWS alignment (Caseware stack): ALB + API Gateway, ElastiCache Redis, RDS (permissions), CloudWatch metrics/logs, X-Ray tracing.

3. Testing Strategy

Layer	Focus
Unit	Scope/claim authorization handlers, cache key construction, token exchange validation rules
Integration	WebApplicationFactory with signed JWTs — 200 / 401 / 403 paths
Contract	Resource APIs reject tokens missing required scope or stale perm_version
Load (design target)	Cache hit ratio under ~10k authz checks/sec; p99 latency budget < 10 ms at API
Security	Confused-deputy scenarios: mismatched sub, overly broad scope, unregistered actor client

The included code slice implements integration tests for Option A (valid scope, missing token, wrong scope, expired token, wrong audience).

4. Evaluation & Observability

Metrics: authz check latency (p50/p99), cache hit rate, token validation failures by reason, revocation propagation lag (time from DB change to cache update).

Logging: Structured fields — sub, client_id, workspace_id, decision (allow/deny). No PII or token values in logs.

Tracing: Correlate login → token issue → resource access → on-behalf-of exchange in a single trace (X-Ray/OpenTelemetry).

Alerts: Spikes in 401/403 rates, cache miss rate above threshold, revocation lag exceeding SLA (target: seconds).

5. Failure Modes & Tradeoffs

Tradeoff	Choice	Risk / Mitigation
Cache vs consistency	Eventual consistency (~seconds) via version + invalidation	Stale allow → perm_version in token; fail closed on mismatch
Token TTL	Short access tokens (5–15 min)	More refresh traffic; avoids mass re-auth on revocation
Scope granularity	Resource-type scopes (documents:read) + claims for overrides	Larger tokens; reference tokens + introspection if needed
Framework vs custom crypto	ASP.NET Core JWT bearer + policy handlers	Built-in handles signatures, clock skew, JWKS rotation; custom code only for domain rules
Redis outage	Fail closed for external users	Higher DB load on fallback path; alert and scale read replicas

Option A implementation note: The code slice uses ASP.NET Core's `JwtBearer` middleware and a custom `ScopeAuthorizationHandler` — no hand-rolled cryptography. Production would use asymmetric keys (RS256) from IdP JWKS; the dev slice uses a symmetric key stub.

Assumptions

- Caseware central IdP exposes OIDC discovery (`/.well-known/openid-configuration`).
- Permission DB emits change events (outbox, CDC, or domain events).
- Redis is available with sub-millisecond latency in-region.
- Access tokens are JWTs enriched at issuance by Collaborate's Token Service.